Trigger nelle Basi di Dati

Paradigma E-C-A, Tipi, Sintassi MySQL ed Esempi

IIS Fermi Sacconi Ceci

Prof. Fedeli Massimo – Basi di Dati

20 marzo 2026

Indice

1. Introduzione
2. Il Paradigma E-C-A
3. Tipi di Trigger
4. I Trigger SQL:2003
5. Trigger in MySQL
6. Privilegi e Gestione degli Errori
7. Esempi Pratici
8. Riepilogo

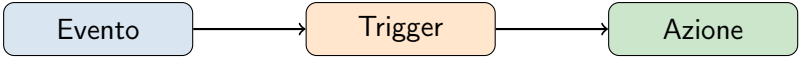
Introduzione ai Trigger

Definizione

L'introduzione di **trigger** all'interno di una Base di Dati permette la **gestione automatica** di particolari procedure in risposta a determinati eventi esterni.

Basi di Dati Attive

Basi di Dati che utilizzano trigger vengono dette **Basi di Dati attive**: i trigger rendono *reattivo* il comportamento del sistema alle sollecitazioni esterne.

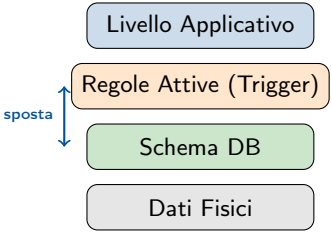


Il Livello di Astrazione dei Dati

L'utilizzo di **regole attive** in un DB aumenta il livello di astrazione dei dati, introducendo una nuova dimensione:

- **Indipendenza della conoscenza**: nasconde all'esterno l'esecuzione delle regole attive.

Il vantaggio principale è lo **spostamento** di alcuni controlli e procedure automatiche **dal livello applicativo** a quello dello schema.



Il Paradigma E-C-A

Il funzionamento dei trigger segue il paradigma **Evento – Condizione – Azione**

Evento

Qualsiasi modifica DML (`INSERT`, `UPDATE`, `DELETE`) o l'attivazione di un altro trigger (*trigger in cascata*) su una *tabella di target*.

Condizione [opz.]

Un predicato booleano espresso mediante sintassi SQL. Se non verificato, il trigger non esegue l'azione.

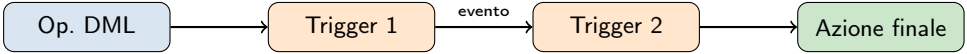
Azione

Sequenza di primitive SQL, eventualmente arricchite da un linguaggio di programmazione integrato nel DBMS (es. PL/SQL, T-SQL).

Trigger in Cascata

Trigger in cascata

Quando l'evento che attiva un trigger è a sua volta l'azione di un altro trigger, si parla di **trigger in cascata**.



Attenzione

I trigger in cascata devono essere progettati con attenzione per evitare cicli infiniti di attivazione.

Tipi di Trigger – Panoramica

Classificazione	Tipo	Descrizione breve
Granularità	Row-level	Eseguito per ogni riga coinvolta
	Statement-level	Eseguito una sola volta per istruzione
Momento	BEFORE	Prima dell'evento DML
	AFTER	Dopo l'evento DML
	INSTEAD OF	Al posto dell'evento (Oracle)
Livello	Tabella	Risponde a eventi DML su tabelle
	Schema/DB	Risponde a eventi DDL

Trigger a Livello di Tupla (Row-Level)

Definizione

I trigger **a livello di riga** (row-level) vengono eseguiti **una volta per ogni riga** su cui agisce un'istruzione DML.

Esempio

Un'istruzione `UPDATE` che modifica 500 righe attiva il trigger **500 volte** – una per ciascuna riga modificata.

Uso tipico:

- Validazione per singola riga
- Calcoli derivati riga per riga
- Sincronizzazione dati distribuiti
- Audit trail dettagliato

Parola chiave MySQL:

```
FOR EACH ROW
```

Obbligatoria in MySQL – solo row-level è supportato.

Trigger a Livello di Istruzione (Statement-Level)

Definizione

I trigger **a livello di istruzione** vengono eseguiti **una sola volta** per ogni istruzione DML, indipendentemente dal numero di righe coinvolte.

Esempio

Se una singola istruzione `INSERT` inserisce 500 righe, un trigger a livello di istruzione viene eseguito **una sola volta**.

Nota

I trigger a livello di istruzione **non vengono utilizzati di frequente** per attività correlate ai singoli dati. Sono più adatti per operazioni di logging a livello globale di transazione.
MySQL non supporta statement-level trigger (`FOR EACH ROW` è obbligatorio).

Trigger BEFORE e AFTER

BEFORE Trigger

- Eseguito **prima** dell'operazione DML
- Può **modificare** i valori `NEW` prima che vengano scritti
- Se fallisce, l'operazione DML **non viene eseguita**
- Utile per: validazione, normalizzazione dati

AFTER Trigger

- Eseguito **dopo** l'operazione DML
- I dati sono già stati modificati
- Viene eseguito solo se BEFORE e DML hanno avuto successo
- Utile per: audit, aggiornamenti correlati, notifiche

Trigger INSTEAD OF

Definizione

In alcuni DBMS (ad es. **Oracle**) è possibile utilizzare trigger `INSTEAD OF` per indicare cosa fare **al posto** di eseguire le azioni che hanno invocato il trigger.

Caso d'uso principale: aggiornamento di viste

Con un trigger `INSTEAD OF` è possibile specificare al DBMS come eseguire `UPDATE`, `DELETE` e `INSERT` nelle **tabelle base della vista** quando un utente tenta di modificare i valori attraverso la vista stessa.

Supporto in MySQL

MySQL **non supporta** i trigger `INSTEAD OF`. Sono disponibili in Oracle, SQL Server e PostgreSQL.

Trigger a Livello di Schema e Database

Definizione

Trigger che si innescano su operazioni eseguite a **livello di schema**:

Esempi di eventi DDL:

- `CREATE TABLE`
- `ALTER TABLE`
- `DROP TABLE`
- `RENAME`
- `TRUNCATE`
- `REVOKE` / `AUDIT`

Capacità principali:

1. **Impedire** l'esecuzione di operazioni DDL non autorizzate
2. **Controllo della sicurezza** aggiuntivo quando si verificano operazioni DDL
3. **Logging** delle modifiche strutturali

I Trigger nello Standard SQL:2003

Definizione formale

Un trigger è una **specifica per una determinata azione** che deve essere eseguita ogniqualvolta avviene un'azione su un determinato oggetto.

Terminologia dello standard:

triggered action L'azione d'innesco: una dichiarazione di procedura SQL o una lista di dichiarazioni.

target table La tabella della base di dati persistente su cui il trigger è definito (tabella obiettivo).

event trigger L'evento scatenante: inserimento, cancellazione o modifica di un insieme di righe.

Per momento	AFTER TRIGGER, BEFORE TRIGGER
Per evento	DELETE, INSERT, UPDATE TRIGGER

Tabelle di Transizione

Scopo

Per ogni trigger possono essere usate **tabelle di transizione** per memorizzare lo stato *precedente* e *successivo* del target.

DELETE Trigger

Solo **old transition table**: contiene le righe prima della cancellazione.

INSERT Trigger

Solo **new transition table**: contiene le righe appena inserite.

UPDATE Trigger

Entrambe le tabelle: old (prima) e new (dopo la modifica).

Evento	OLD	NEW
INSERT	×	✓
DELETE	✓	×

La Granularità dell'Azione

L'azione del trigger può essere di tipo:

Statement-Level Trigger

Eseguito **solo una volta** dopo l'attivazione del trigger, indipendentemente dal numero di righe coinvolte.

- La variabile di transizione ha il valore dell'intera tabella

Row-Level Trigger

Eseguito **una volta per ogni riga** della tabella di transizione associata al trigger.

- La variabile di transizione assume il valore della singola tupla

Variabili dei Trigger

Vengono messe a disposizione dei trigger variabili speciali per accedere alle tabelle di transizione.

Statement-Level Trigger

La variabile ha il valore dell'intera **tabella di transizione**.

Row-Level Trigger

La **variabile di transizione** associa ad ogni azione del trigger il valore della tupla nella tabella di transizione da modificare.

Tipi di variabile (row-level)

- **old transition variable**: valore della tupla *prima* della modifica (DELETE, UPDATE)
- **new transition variable**: valore della tupla *dopo* la modifica (INSERT, UPDATE)

Definizione Completa di un Trigger

Un trigger è definito dai seguenti componenti:

1. **Nome** del trigger
2. **Tabella di target**
3. **Momento d'azione**: `BEFORE` / `AFTER`
4. **Evento**: `INSERT` / `UPDATE` / `DELETE`
5. **Granularità**: statement-level / row-level
6. **Nomi** per le tabelle di transizione o variabili
7. **L'azione** SQL da eseguire
8. **Timestamp** della creazione

Schema sintetico

`CREATE TRIGGER` *nome* `BEFORE|AFTER INSERT|UPDATE|DELETE ON` *tabella* `FOR EACH ROW` *azione*

Contesto di Esecuzione del Trigger (TEC)

Esecuzione

Durante l'esecuzione di un'istruzione S possono esistere **zero o più** contesti d'esecuzione trigger (TEC), ma **al più uno** di loro può essere attivo alla volta.

Ciclo di vita del TEC

Il contesto di esecuzione di un trigger (TEC) viene:

1. **Creato** al momento dell'evento scatenante
2. **Attivato** se la condizione è verificata
3. **Eseguito**: viene eseguita l'azione definita
4. **Distrutto** al termine dell'azione

Sintassi CREATE TRIGGER in MySQL

```
CREATE
  [DEFINER = { user | CURRENT_USER }]
  TRIGGER nome_trigger
  { BEFORE | AFTER }
  { INSERT | UPDATE | DELETE }
  ON tabella_target
  FOR EACH ROW
    trigger_stmt;
```

- **DEFINER**: account utente i cui privilegi vengono verificati all'attivazione
- **BEFORE | AFTER**: momento di esecuzione
- **INSERT | UPDATE | DELETE**: evento scatenante
- **FOR EACH ROW**: granularità (obbligatorio in MySQL)
- **trigger_stmt**: corpo del trigger

Parametri della Sintassi – Dettaglio

`DEFINER` Determina i privilegi da applicare quando il trigger è attivo. Formato: `'user_name'@'host_name'`

`TRIGGER_TIME` Rappresenta il tipo di trigger: `BEFORE` o `AFTER`

`TRIGGER_EVENT` Indica il tipo di primitiva che attiva il trigger:

- `INSERT`: trigger attivato su `INSERT`, `LOAD DATA`, `REPLACE`
- `UPDATE`: trigger attivato su `UPDATE`
- `DELETE`: trigger attivato su `DELETE`, `REPLACE`

`TRIGGER_STMT` La primitiva SQL da eseguire. Per sequenze multiple si usa il blocco `BEGIN ...END`

Le Variabili OLD e NEW

Per accedere alle colonne della tabella target si usano gli alias **OLD** e **NEW**:

`OLD.col_name` (read-only)

Si riferisce alla colonna della tabella target **prima** della modifica effettuata dall'evento associato al trigger.

`NEW.col_name`

Si riferisce alla colonna della tabella target **dopo** la modifica effettuata dall'evento.

Tipo Trigger	OLD disponibile	NEW disponibile
INSERT	No	Sì
DELETE	Sì	No
UPDATE	Sì	Sì

Modifica di NEW in un BEFORE Trigger

Caratteristica speciale dei BEFORE Trigger

In un BEFORE trigger è possibile **modificare** i valori che verranno scritti nella tabella usando:

```
SET NEW.colonna = valore
```

Esempio: normalizzazione automatica

```sql
-- Questo BEFORE INSERT trigger
-- capitalizza automaticamente il nome
CREATE TRIGGER normalizza_nome
BEFORE INSERT ON clienti
FOR EACH ROW
  SET NEW.nome = CONCAT(
    UPPER(LEFT(NEW.nome, 1)),
    LOWER(SUBSTRING(NEW.nome, 2))
  );
```

Privilegi per la Creazione di Trigger

Al momento della creazione

L'utente che crea il trigger deve avere i **privilegi di SuperUtente**.

Clausola DEFINER

Specifica l'account utente associato al trigger. Formato:

```
'user'@'host'
```

I privilegi dell'account DEFINER vengono verificati all'attivazione.

Privilegi necessari all'attivazione

L'utente DEFINER deve possedere:

- Privilegi di **SuperUtente**
- **SELECT** sulla tabella target (se usa OLD/NEW)
- **UPDATE** sulla tabella target (se usa `SET NEW.col`)
- Tutti i privilegi per gli statement SQL nell'azione

Gestione degli Errori nei Trigger

Regole di esecuzione

1. Se un **BEFORE trigger fallisce**, l'operazione DML che ha generato l'evento **non viene eseguita**.
2. Un **AFTER trigger viene eseguito** solo se i BEFORE trigger e l'operazione DML hanno avuto successo.
3. Un **errore** durante un BEFORE o AFTER trigger genera il fallimento dell'intera procedura.

Sistema transazionale

Il fallimento causa il **rollback** di *tutte* le modifiche effettuate dal trigger.

Sistema non transazionale

Tutte le modifiche apportate *prima* dell'errore **rimangono effettive**.

Limitazioni dei Trigger in MySQL

Unicità per evento

In MySQL **non possono essere creati due trigger** con evento e tipo evento uguali sulla stessa tabella target.

Esempi di combinazioni non ammesse

Non possono coesistere sulla stessa tabella:

- Due trigger `BEFORE INSERT`
- Due trigger `AFTER UPDATE`
- Due trigger `BEFORE DELETE`

Altre limitazioni

- I trigger **non possono invocare** procedure che restituiscono dati all'utente o che usano SQL dinamico

Chiamata di Procedure nei Trigger

Procedure NON invocabili

- Procedure che **restituiscono dati** all'utente
- Procedure che usano **SQL dinamico**
- Procedure con `START TRANSACTION`
- Procedure con `END TRANSACTION`
- Procedure con `COMMIT`
- Procedure con `ROLLBACK`

Costrutti utilizzabili

All'interno del corpo del trigger è possibile utilizzare:

- Blocco `BEGIN ...END`
- `IF ...THEN ...ELSE`
- `DECLARE` per variabili locali
- `SELECT ...INTO`
- `INSERT`, `UPDATE`, `DELETE`
- Chiamata a **funzioni** definite dall'utente

Esempio Semplice: Trigger su INSERT

```sql
-- Creazione tabella
CREATE TABLE account (
  acct_num INT,
  amount   DECIMAL(10,2)
);

-- Trigger: accumula la somma degli importi inseriti
CREATE TRIGGER ins_sum
BEFORE INSERT ON account
FOR EACH ROW
  SET @sum = @sum + NEW.amount;
```

Utilizzo

```sql
SET @sum = 0;
INSERT INTO account VALUES (1, 14.98), (2, 1937.50);
SELECT @sum;  -- Risultato: 1952.48
```

Esempio 1 – Tabella Utente

```sql
CREATE TABLE `bd2`.`utente` (
  `idutente`  INTEGER UNSIGNED NOT NULL AUTO_INCREMENT,
  `nome`      VARCHAR(15) NOT NULL DEFAULT '',
  `cognome`   VARCHAR(25) NOT NULL DEFAULT '',
  `password`  VARCHAR(45) NOT NULL DEFAULT '',
  PRIMARY KEY (`idutente`)
) ENGINE = InnoDB;
```

Obiettivo del trigger

Ogni volta che viene inserito un utente:

1. Il nome e il cognome vengono **capitalizzati** automaticamente
2. La password viene **hashata** con la funzione `PASSWORD()`

Esempio 1 – Funzione di Validazione

```
DELIMITER //
CREATE FUNCTION valida_stringa (str VARCHAR(25))
RETURNS VARCHAR(25)
DETERMINISTIC
BEGIN
  DECLARE testa VARCHAR(1);
  DECLARE coda  VARCHAR(24);

  SET str   = TRIM(str);
  SET testa = UPPER(LEFT(str, 1));
  SET coda  = LOWER(RIGHT(str, LENGTH(str) - 1));

  RETURN CONCAT(testa, coda);
END;
//
DELIMITER ;
```

Comportamento

Esempio 1 – Creazione del Trigger

```
DELIMITER //
CREATE TRIGGER UTENTE_BEF_INS_ROW
BEFORE INSERT ON utente
FOR EACH ROW
BEGIN
  SET NEW.nome     = valida_stringa(NEW.nome);
  SET NEW.cognome  = valida_stringa(NEW.cognome);
  SET NEW.password = PASSWORD(NEW.password);
END;
//
DELIMITER ;
```

Nota sulla convenzione di naming

Il nome `UTENTE_BEF_INS_ROW` codifica: tabella (`UTENTE`), momento (`BEF`), evento (`INS`), granularità (`ROW`).

Esempio 1 – Inserimento e Risultato

```sql
-- Inserimento con dati non normalizzati
INSERT INTO utente VALUES (1, 'alessio',    'meola',       'passwd1010');
INSERT INTO utente VALUES (2, 'gIuSePPe',   'MAZZARELLA',  'passwd2020');
```

Risultato dopo SELECT * FROM utente

```
+----------+----------+------------+----------------------------------+
| idutente | nome     | cognome    | password                         |
+----------+----------+------------+----------------------------------+
|        1 | Alessio  | Meola      | *4BD714D8E708D3D6BF85BB246E0DCE6C32|
|        2 | Giuseppe | Mazzarella | *80BCC99B46B82A5A8C18B98EDD7585AE51|
+----------+----------+------------+----------------------------------+
```

Osservazioni

Il trigger ha automaticamente capitalizzato i nomi e hashato le password, indipendentemente dal formato di input fornito.

Esempio 2 – Gestione Magazzino

Scenario

Un sistema di gestione magazzino deve gestire automaticamente:

1. **Riordino della merce**: quando la quantità scende sotto il limite minimo, viene creato automaticamente un ordine
2. **Statistiche sulle vendite**: aggiornamento automatico delle analisi per prodotto

Soluzione implementata

- **2 trigger**: `PROD_MAG_AFT_UPD_ROW` e `PROD_MAG_AFT_INS_ROW`
- **1 funzione**: `analisi_vendite()`

Esempio 2 – Schema delle Tabelle

```sql
-- Prodotti disponibili
CREATE TABLE prodotto (
  idprodotto INTEGER UNSIGNED NOT NULL AUTO_INCREMENT,
  nome       VARCHAR(45) NOT NULL,
  marca      VARCHAR(45) NOT NULL,
  vendita    FLOAT       NOT NULL DEFAULT 0,  -- prezzo vendita
  acquisto   FLOAT       NOT NULL DEFAULT 0,  -- prezzo acquisto
  PRIMARY KEY(idprodotto)
) ENGINE = InnoDB;

-- Depositi/Magazzini
CREATE TABLE magazzino (
  idmagazzino INTEGER UNSIGNED NOT NULL AUTO_INCREMENT,
  citta       VARCHAR(45) NOT NULL,
  PRIMARY KEY(idmagazzino)
) ENGINE = InnoDB;
```

Esempio 2 – Tabella Prodotti_Magazzino

```
CREATE TABLE prodotti_magazzino (
  idprodotto   INTEGER UNSIGNED NOT NULL AUTO_INCREMENT,
  idmagazzino  INTEGER UNSIGNED NOT NULL DEFAULT 0,
  qta          INTEGER UNSIGNED NOT NULL DEFAULT 0,
  qta_limite   INTEGER UNSIGNED NOT NULL DEFAULT 0,
  qta_riordino INTEGER UNSIGNED NOT NULL DEFAULT 0,
  PRIMARY KEY(idprodotto, idmagazzino),
  CONSTRAINT prod_mag_prod FOREIGN KEY (idprodotto)
    REFERENCES prodotto(idprodotto)
    ON DELETE RESTRICT ON UPDATE RESTRICT,
  CONSTRAINT prod_mag_mag FOREIGN KEY (idmagazzino)
    REFERENCES magazzino(idmagazzino)
    ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE = InnoDB;
```

Chiave

qta_limite: soglia minima sotto cui scatta il riordino.

qta_riordino: quantità da ordinare al fornitore.

Esempio 2 – Tabelle Ordine e Analisi

```sql
-- Ordini di rifornitamento generati automaticamente
CREATE TABLE ordine (
  idprodotto  INTEGER UNSIGNED NOT NULL AUTO_INCREMENT,
  idmagazzino INTEGER UNSIGNED NOT NULL DEFAULT 0,
  data        DATETIME         NOT NULL DEFAULT 0,
  PRIMARY KEY(idmagazzino, idprodotto),
  CONSTRAINT ord_prod FOREIGN KEY (idprodotto)
    REFERENCES prodotto(idprodotto),
  CONSTRAINT ord_mag FOREIGN KEY (idmagazzino)
    REFERENCES magazzino(idmagazzino)
) ENGINE = InnoDB;

-- Statistiche di vendita per prodotto
CREATE TABLE analisi (
  idprodotto INTEGER UNSIGNED NOT NULL AUTO_INCREMENT,
  venduto    INTEGER UNSIGNED NOT NULL DEFAULT 0,
  guadagno   FLOAT            NOT NULL DEFAULT 0,
  PRIMARY KEY(idprodotto),
  CONSTRAINT analisi_prod FOREIGN KEY (idprodotto)
```

Esempio 2 – Funzione analisi_vendite()

```
DELIMITER //
CREATE FUNCTION analisi_vendite(prodotto INT, qta INT)
RETURNS INTEGER
DETERMINISTIC
BEGIN
  DECLARE guad INTEGER;

  -- Calcola il guadagno unitario (vendita - acquisto)
  SELECT (vendita - acquisto) INTO guad
  FROM prodotto WHERE idprodotto = prodotto;

  -- Aggiorna le statistiche
  UPDATE analisi
  SET venduto  = venduto + qta,
      guadagno = guadagno + (guad * qta)
  WHERE idprodotto = prodotto;

  RETURN 1;
END;
//
```

Esempio 2 – Trigger Riordino (AFTER UPDATE)

```
DELIMITER //
CREATE TRIGGER PROD_MAG_AFT_UPD_ROW
AFTER UPDATE OF qta ON prodotti_magazzino
FOR EACH ROW
BEGIN
  DECLARE nordini    INTEGER;
  DECLARE qtavenduta INTEGER;

  SET qtavenduta = OLD.qta - NEW.qta;  -- quantita' venduta

  -- Controlla se la scorta e' sotto il limite
  IF NEW.qta < NEW.qta_limite THEN
    SELECT COUNT(*) INTO nordini FROM ordine
    WHERE ordine.idprodotto  = NEW.idprodotto
      AND ordine.idmagazzino = NEW.idmagazzino;

    -- Crea ordine solo se non ne esiste gia' uno
    IF nordini = 0 THEN
      INSERT INTO ordine VALUES
```

Esempio 2 – Trigger Inizializzazione Analisi

```
-- Trigger: inizializza il record in "analisi"
-- quando un prodotto viene aggiunto al magazzino
CREATE TRIGGER PROD_MAG_AFT_INS_ROW
AFTER INSERT ON prodotti_magazzino
FOR EACH ROW
BEGIN
  IF NOT EXISTS (
    SELECT * FROM analisi
    WHERE idprodotto = NEW.idprodotto
  ) THEN
    INSERT INTO analisi VALUES (NEW.idprodotto, 0, 0);
  END IF;
END;
//
DELIMITER ;
```

Logica

Quando un prodotto viene inserito in magazzino per la prima volta, viene creato

Esempio 2 – Esecuzione e Risultati

```sql
-- Vendita: riduzione scorte prodotto 3, magazzino 1
UPDATE prodotti_magazzino
SET qta = qta - 41
WHERE idprodotto = 3 AND idmagazzino = 1;
```

Tabella ordine (auto-generato)

```
+----+-----+------------+
|prod|mag  |data        |
+----+-----+------------+
|  3 |  1  |2006-01-13  |
|  1 |  2  |2006-01-13  |
+----+-----+------------+
```

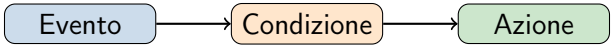
Tabella analisi (aggiornata)

```
+----+--------+---------+
|prod|venduto |guadagno |
+----+--------+---------+
|  1 |      8 |     400 |
|  2 |      0 |       0 |
|  3 |     41 |    3280 |
+----+--------+---------+
```

Riepilogo – Trigger in MySQL

Punti chiave

- I trigger implementano il paradigma **E-C-A**
- Le basi di dati con trigger sono dette **attive**
- MySQL supporta solo trigger **row-level**
- Le variabili **OLD** e **NEW** accedono ai dati
- **BEFORE**: modifica dati prima della scrittura
- **AFTER**: azioni collaterali post-scrittura

Best Practice

- Naming convention chiara (tabella_BEF/AFT_INS/UPD/DEL_ROW)
- Evitare logiche complesse nei trigger
- Attenzione ai trigger in cascata
- Testare il comportamento su errori
- Un solo trigger per combinazione evento/tempo/tabella